🧠 What is Inheritance?

Inheritance allows a class (called the **child** or **subclass**) to **inherit attributes and methods** from another class (called the **parent** or **base class**).

✅ It helps **reuse code** and makes programs more **modular and maintainable**.

🧩 Basic Example

```
# Parent Class
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print("Animal makes a sound")

# Child Class inherits from Animal
class Dog(Animal):
    def speak(self):  # Overriding parent method
        print(f"{self.name} says Woof!")

# Create object of child class
dog = Dog("Buddy")
dog.speak()   # Output: Buddy says Woof!
```

🧍 Explanation:

- Dog inherits from Animal.
- It gets access to the __init__() and speak() methods.
- But it **overrides** the speak() method.

🔁 Using super() to call parent methods

You can call the parent class's methods using super().

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)   # Call parent constructor
        self.breed = breed

    def display(self):
        print(f"{self.name} is a {self.breed}")

dog = Dog("Buddy", "Golden Retriever")
dog.display()
```

🧱 Types of Inheritance in Python

Type	Description	Example
Single	One child inherits from one parent	class Dog(Animal)
Multiple	Child inherits from multiple parents	class Puppy(Dog, Cat)
Multilevel	One class inherits from another child class	A -> B -> C
Hierarchical	Multiple children inherit from same parent	Dog and Cat inherit Animal
Hybrid	Combination of above types	Complex structure

🧩 **Example: Multiple Inheritance**

```
class Father:
    def skills(self):
        print("Father: Gardening, Programming")

class Mother:
    def skills(self):
        print("Mother: Cooking, Painting")

class Child(Father, Mother):
    def skills(self):
        super().skills()
        print("Child: Dancing, Singing")

obj = Child()
obj.skills()
```

🧠 **Output (uses Method Resolution Order - MRO):**

```
Father: Gardening, Programming
Child: Dancing, Singing
```

🧭 **Checking Inheritance Info**

```
print(issubclass(Dog, Animal))  # True
print(isinstance(dog, Animal))  # True
```

🧰 **When to Use Inheritance**

✅ When classes share **common behavior**

✅ When you want to **extend or customize** functionality

❌ Avoid when classes are **unrelated** — prefer **composition** instead

---

🧩 1️⃣ **Single Inheritance**

👉 **One child class inherits from one parent class**

📘 **Example:**

```
class Animal:
    def speak(self):
        print("Animal makes a sound")


class Dog(Animal):   # Single inheritance
    def bark(self):
        print("Dog barks")


# Create object
d = Dog()
d.speak()   # Inherited method
d.bark()    # Child's own method
```

🖥️ **Output:**

```
Animal makes a sound
Dog barks
```

Diagram:

Animal → Dog

🧩 2️⃣ **Multiple Inheritance**

👉 **A class inherits from more than one parent.**

📘 **Example:**

```
class Father:
    def skill(self):
        print("Father: Driving")


class Mother:
    def skill(self):
        print("Mother: Cooking")


class Child(Father, Mother):   # Multiple inheritance
    def skill(self):
        super().skill()  # Calls the first parent as per MRO
        print("Child: Dancing")


obj = Child()
obj.skill()
```

🖥️ **Output:**

```
Father: Driving
Child: Dancing
```

Diagram:

```
 Father
   ↓
 Child
   ↑
```

Mother

Note:

- The order of inheritance matters.
- Python uses **MRO (Method Resolution Order)** to decide which parent method to call first.

🧩 3️⃣ **Multilevel Inheritance**

👉 **In this type, a child class becomes a parent for another child class.**

📘 **Example:**

```
class Grandfather:
    def show(self):
        print("Grandfather: Family Head")


class Father(Grandfather):   # inherits Grandfather
    def show(self):
        super().show()
        print("Father: Next Generation")


class Son(Father):   # inherits Father
    def show(self):
        super().show()
        print("Son: Youngest Generation")


obj = Son()
obj.show()
```

🖥️ **Output:**

Grandfather: Family Head

Father: Next Generation

Son: Youngest Generation

Diagram:

Grandfather → Father → Son

🧩 🛗 **Hierarchical Inheritance**

👉 **Multiple child classes inherit from a single parent.**

📘 **Example:**

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Dog(Animal):
    def speak(self):
        print("Dog barks")

class Cat(Animal):
    def speak(self):
        print("Cat meows")

# Create objects
d = Dog()
c = Cat()

d.speak()
```

c.speak()

🖥️ **Output:**

Dog barks

Cat meows

Diagram:

```
    Animal
    /   \
  Dog    Cat
```

🧩 5️⃣ **Hybrid Inheritance**

👉 **Combination of two or more inheritance types (like multiple + multilevel).**

📘 **Example:**

```
class School:
    def show_school(self):
        print("This is ABC School")


class Student(School):
    def show_student(self):
        print("This is a student")


class Sports:
    def show_sport(self):
        print("Plays Football")


# Hybrid: Player inherits from both Student and Sports
class Player(Student, Sports):
```

```
    def show_player(self):
        print("This student is also a player")


p = Player()
p.show_school()   # From School (via Student)
p.show_student()  # From Student
p.show_sport()    # From Sports
p.show_player()   # Own method
```

🖥️ **Output:**

```
This is ABC School
This is a student
Plays Football
This student is also a player
```

Diagram:

```
    School      Sports
      \       /
       \     /
        Student
          ↓
        Player
```

🧭 **Summary Table**

Type	Description	Example Classes
Single	One parent → one child	Animal → Dog
Multiple	Multiple parents → one child	Father, Mother → Child

Type	Description	Example Classes
Multilevel	Parent → Child → Grandchild	Grandfather → Father → Son
Hierarchical	One parent → multiple children	Animal → Dog, Cat
Hybrid	Combination of multiple types	School, Sports → Player (via Student)